

Build a Customer Support Agent using OpenAI and AzureML

Project Overview

Overview

The project tackles the significant business problem of efficiently managing a large influx of customer support tickets, which often results in delayed responses, misprioritization, and inconsistent quality in customer service. Traditional support systems struggle to scale, leading to increased operational costs and customer dissatisfaction. By leveraging Large Language Models (LLMs), this project aims to revolutionize the customer support process. LLMs can analyze and understand the content of support tickets, categorize them based on urgency and subject matter, and generate accurate and contextually appropriate responses.

In this project, we are developing a system that will use LLMs for tasks such as sentiment analysis, ticket categorization, and response generation, ensuring that critical issues are prioritized and handled promptly. Azure ML will provide the infrastructure needed to deploy, scale, and manage the AI models efficiently.

The problem statement centers around the need to reduce the costs associated with manual ticket handling, improve response times, and enhance customer satisfaction. By automating these processes, companies can significantly lower operational expenses related to support staff and improve the overall customer experience.

Prerequisite Project: Kindly ensure the completion of the [LLM Project for building and fine-tuning a large language model](#) before proceeding with this project.

Prerequisites: Basics of LLMs, Vector Databases, Prompt Engineering, Python and Streamlit

Note: Utilizing Azure services for this project may result in charges; it is essential to thoroughly review the Azure documentation to understand the pricing structure and potential costs associated with different resources and usage patterns.

Aim

The aim of the project is to enhance customer support efficiency and reduce operational costs by leveraging Large Language Models (LLMs) and Azure Machine Learning for automated ticket categorization, prioritization, and response generation.

Data Description

The retail CSV dataset includes customer support ticket data encompassing instructions categorized by issue type and corresponding responses.

Tech Stack

- Language: Python 3.10
- Libraries: pandas, langchain, langchain-openai, openai faiss-cpu, streamlit
- Model: Open-AI Embeddings and GPT-4
- Cloud Platform: Azure

→

Approach

- **Azure ML Workspace Setup**
 - Configure Azure ML workspace and connect it to your local development environment.
- **Data Loading and Analysis**
 - Load the retail dataset containing customer support tickets and analyze.
- **LLM Integration**
 - Integrate a pre-trained LLM for generating embeddings and responses.
- **Vector Database Setup**
 - Set up a vector database (FAISS) to store embeddings for efficient retrieval.
- **Prompt Engineering**
 - Develop and refine prompting strategies to generate accurate and contextually relevant responses.
- **Retrieval-Augmented Generation (RAG) Implementation**
 - **RAG Architecture**
 - Implement the RAG framework to combine retrieval-based and generation-based approaches.
 - Use the vector database for retrieving relevant responses based on customer query embeddings.
 - **Response Generation**
 - Implement the logic for generating responses using the retrieved information and the LLM.
- **Response Sampling**
 - Implement a sampling mechanism to generate multiple response candidates.
 - Provide an interface for selecting the best response from the generated options.
- **Feedback Loop**
 - Creating a Feedback loop to improve the response based on prompt improvements.

- **Code Modularity and Streamlit UI**
 - Modularize the code to ensure easy integration and create a Streamlit UI
- **Azure Deployment**
 - Deploy the application on Azure ML, ensuring it is scalable and easy to maintain.

Cost Breakdown

Components:

- Azure ML Instance with storage account
- Azure App Service Deployment
- Open AI API Calls

Cost Breakdown as of January 17, 2025

The cost might have been updated, it is recommended to go through the following pricing pages to come up with updated guessestimates.

Open AI Pricing <https://openai.com/api/pricing/>

Azure App service Pricing - [Pricing Page](#)

Azure Blob Storage - <https://azure.microsoft.com/en-us/pricing/details/storage/blobs/>

Component	Description	Usage	Estimated Cost
API Calls	OpenAI GPT-4o API for ticket categorization & response generation	100 API calls (1,000 input + 500 output tokens per call)- Input: \$5 per 1M tokens- Output: \$15 per 1M tokens	~\$1.25
Compute Resources	Azure App Service deployment (B1 plan)	10 hours of usage- Monthly cost: \$54.75 for 730 hours- Cost per hour: ~\$0.075	~\$0.76
Storage	Azure Blob Storage for customer support ticket data and embeddings	10 GB- Pricing: \$0.018 per GB/month	~\$0.18

Total Estimated Cost for Experimentation Phase:

- \$2.45 (Approx.) for 100 API calls and 10 hours of deployment.

For this experiment, the total cost is estimated at \$2.03. This includes \$1.25 for OpenAI API calls, \$0.76 for Azure App Service deployment, and \$0.18 for storing customer support ticket data in Azure Blob Storage (using the Hot tier). To put it in simpler terms, if you're processing a small dataset, you can expect a total cost of around \$2.45 which includes the cloud service cost, API usage for generating responses, and storing the data. If the number of customer queries or stored data is low, you might save costs by using a Cold tier for storage. Please note the cost is for experimentation only and for running application for longer times assuming you shut off unnecessary instances and resources.

Recommendation:

- Monitor API usage and optimize prompt engineering to reduce token consumption.
- API calls form the largest portion of the cost; optimizing token usage can reduce expenses.
- Compute costs can be minimized by scaling down during off-peak hours.
- Storage costs are relatively low but can increase with higher data volumes.

Modular code overview:

Once you unzip the modular_code.zip file, you can find the following:

```
├ Customer_Support_Training_Dataset
|   └ Customer_Support_Training_Dataset.csv
|   └ dataset.md
├ analysis.ipynb
├ README.md
├ demo.py
├ src
|   └ helper.py
├ vector_store
|   └ faiss_index.index
└ requirements.txt
```

Here is a brief information on the files:

- The folder contains CSV file, `Customer_Support_Training_Dataset.csv`, which holds training data for customer support-related tasks, alongside a `dataset.md` file that documents the dataset's details.
 - The project includes a Jupyter notebook (`analysis.ipynb`) for data analysis, a Streamlit demo script (`demo.py`), and a `helper.py` file within the `src` folder, which contains utility functions for the project.
 - The `vector_store` folder holds a FAISS index (`faiss_index.index`) used for fast similarity search, and the `requirements.txt` file lists the necessary dependencies for the project.
- Kindly follow all the instructions for running the code from Readme.md file.**

Project Takeaways

1. Recognize the challenges in managing high volumes of customer support tickets and the need for automation
2. Learn to set up and configure an Azure ML workspace for managing and tracking machine learning projects
3. Implement and run AI workflows using Azure ML pipelines
4. Explore how to use Open AI Embeddings to convert text data into embeddings for efficient retrieval
5. Understand the role of vector databases in storing and retrieving embeddings
6. Implement a vector database to store and retrieve text embeddings efficiently
7. Learn the Retrieval-Augmented Generation (RAG) principles and how it combines retrieval and generation techniques
8. Understand how to generate responses using LLMs based on retrieved information
9. Explore techniques for generating multiple response candidates and selecting the best one
10. Build a user-friendly demo application using Streamlit to showcase the AI assistant's capabilities
11. Create a Feedback loop to improve the response based on prompt improvements
12. Learn the steps involved in deploying the AI application on Azure

